

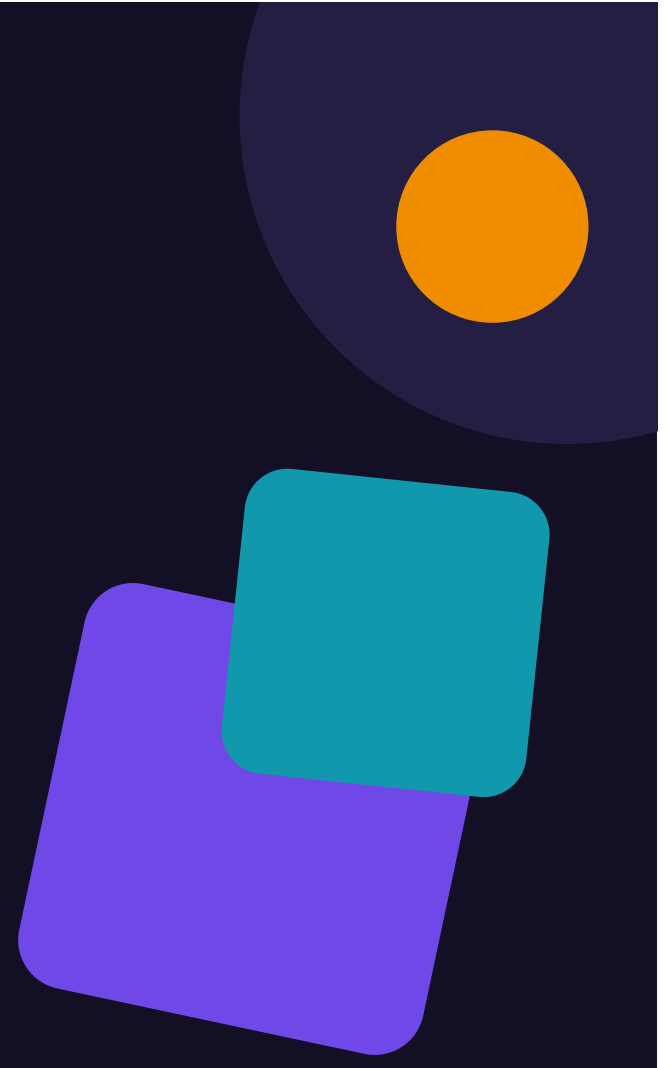
4-HOUR MASTERCLASS · SEPTEMBER 2026

AI Cost Management and Token Utilization

Token economics, context optimization, intelligent routing, and the FinOps loop that keeps AI spend defensible.

Tarek Atwan

AI Engineer & Instructor



Three numbers that should not be true at the same time

Where we are, September 2026

280×

Cheaper GPT-3.5-class inference. From \$20/M tokens in November 2022 to \$0.07/M in October 2024, for a model achieving GPT-3.5 equivalent MMLU performance.

Stanford HAI AI Index 2025 reports. a16z LLMflation (2024)

\$12.5B

Enterprise foundation-model API spend in 2025. Part of \$37B in total Enterprise GenAI spend. API spend rose despite per-unit inference deflation, because production usage expanded much faster.

Menlo Ventures, December 2025 2025: The State of Generative AI in the Enterprise

98%

of FinOps practitioners now manage AI spend — 31% two years ago. FinOps for AI is their #1 forward-looking priority.

FinOps Foundation, State of FinOps 2026 (n=1,192)

Unit prices collapsed. Bills went up.

Jevons Paradox: making tokens cheaper did not lower bills - it made organizations consume more tokens per task

“

AI projects are approved on capability.
They are killed on cost.

Cost surprises now routinely force project cuts. Volatility in token consumption is the silent killer — one model swap, one prompt change, one team doubling its volume, and nobody saw the invoice coming.

78% of IT leaders were hit with unexpected charges from AI or consumption-based pricing models in the past 12 months. **61%** were forced to cut projects or initiatives because of unplanned SaaS cost increases.

Zylo 2026 SaaS Management Index (n=218 IT leaders)

The paradox, resolved

Why cheaper tokens can still produce bigger invoices

WHAT GOT CHEAPER

↓ **50×/yr** Median cross-benchmark price decline

- 9× to 900×/yr on individual tasks
- Cache-hit input can be 50–97.5% cheaper than standard input
- Async batch processing is often ~50% cheaper
- Lower-cost models can handle many bounded tasks

WHAT GOT MORE EXPENSIVE

↑ **5–30×** Mode tokens consumed per task, as chat became agentic

- **More context:** system prompts, retrieved content, and conversation history
- **More calls:** planning, tools, retries, validation, and repair loops
- **More output:** reasoning tokens (billed as output) and generated artifacts
- **More users** and more embedded AI features

This is Jevons' paradox with a login screen. Price per token fell. Tokens per task rose faster. **You are not being overcharged — you are consuming a different product than you budgeted for.**

Epoch AI (2025); Gartner (March 2026) — agentic tasks consume 5–30× the tokens of a standard chatbot exchange; Deloitte, "The Pivot to Tokenomics" (2026).

Two companies, the same arithmetic, opposite endings

The variable was not budget, discipline or engineering quality — it was when governance arrived

UBER · CONTROLS APPLIED AFTER THE SPEND

4 months

Claude Code rolled out December 2025 to ~5,000 engineers

Adoption climbed to 32% Claude Code (Dec to Feb) then by March 84% agentic coding users across all tools

By April the entire 2026 AI budget was gone.

The rollout succeeded on every measure it was designed against: 70% of committed code AI-generated, 11% of backend updates from autonomous agents.

The response was a cap: \$1,500 per employee, per tool, per month.

COINBASE · DEFAULTS DESIGNED BEFORE THE SPEND

Bill halved

Spend cut nearly in half while token usage kept growing.

Method: better defaults, routing and caching in an internal gateway — not friction and spend alerts.

LibreChat's cache hit rate went from 5% to ~50%.

They found 91% of employees never hit their caps anyway — caps solved a problem 9% of the company had.

Uber reached for controls. Coinbase built defaults. The fix is defaults, not caps — a theme we return to in Module 4.

Uber engineering spend reported 2026 (Bloomberg, 2 June 2026); Brian Armstrong, Coinbase, June 2026.

“

Through 2028, at least 50% of GenAI projects will overrun their budgeted costs due to poor architectural choices and lack of operational know-how

Gartner, June 2026 · Inference is projected to reach $\geq 70\%$ of a model's lifetime cost

Read the cause again: **architecture and operations**. Not vendor pricing. Which means it sits inside your control.

What you will be able to do end of session

Four outcomes, one per module

01

Forecast from first principles

Build a defensible monthly cost model from prompt shape and volume — including the three multipliers that break every naive forecast.

02

Cut 30–50% without touching the product

Audit a prompt stack, restructure it for cache hits, right-size retrieval — with a quality gate that proves you did not degrade the answer.

03

Route work to the cheapest capable model

Design a tiered routing policy and know the honest savings range, from the 85% headline down to the ~30% a calibrated cascade delivers in production.

04

Stand up the FinOps loop

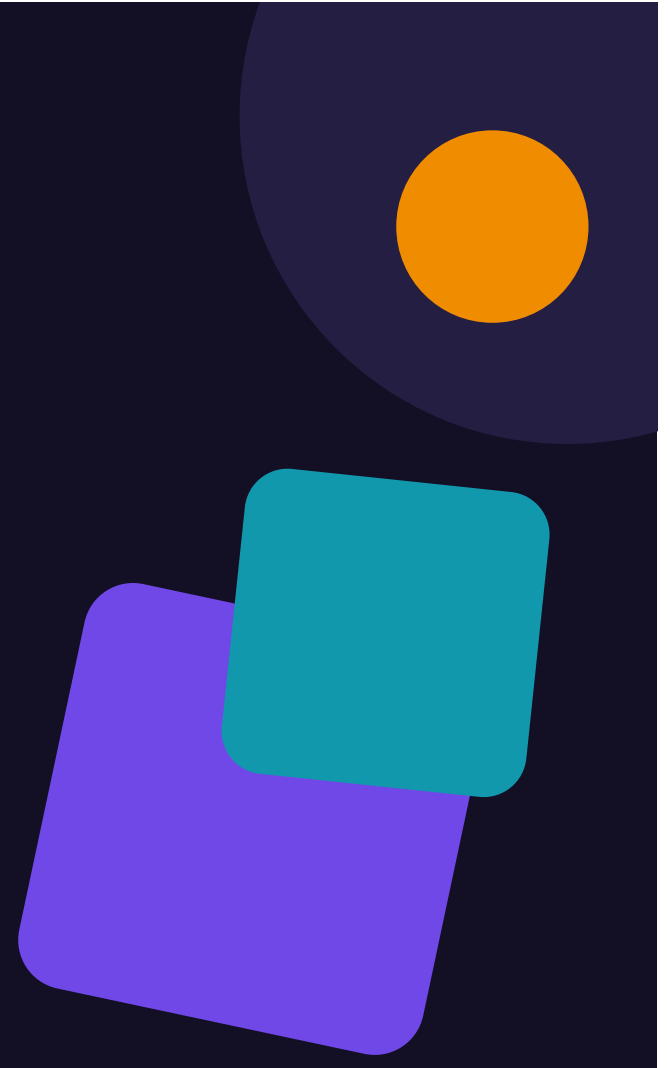
Tag, attribute, cap and report using cost per completed task — the only unit that survives a CFO review of an agentic system.

MODULE 1

Your bill is not a price problem

Token economics and cost forecasting — how a request becomes an invoice, and which three multipliers break every forecast you have ever written.

48 MINUTES · 3 LIVE DEMOS · 1 WORKED EXAMPLE



A token is not a word — and that costs you money

Byte-pair encoding, and why the same idea has four different prices

- 01 **~4 characters, ~¾ of a word in English.** A rule of thumb, not a rule. It breaks the moment your text stops looking like English prose.
- 02 **Common subwords get one token; rare ones shatter.** SKUs, custom field API names, UUIDs and internal jargon fragment into 4–9 tokens each.
- 03 **Code, JSON and non-English text tokenize badly.** Braces, quotes and repeated keys are billed at the same rate as meaning.
- 04 **Tokenizers differ between vendors.** The same string produces different counts on different models, so a \$/1M comparison alone is incomplete.
- 05 **You pay for the form of your prompt, not its meaning.** Which makes token cost a writing decision before it is a model decision.

THE SERIALIZATION TAX

25-60%

- Typical token reduction from moving uniform tabular data out of verbose JSON into a compact row format (Markdown tables, CSV, TOON – minified JSON alone saves ~25-30%)
- JSON repeats every key on every record, then wraps it all in quotes, braces, and commas.
- Note: JSON remains the right format when the model must return or strictly parse structured output, or when data is nested rather than tabular.

Rewrite before you optimise. The cheapest token is the one you never had to send because you phrased the request better.

DEMO

6 min · Module 1

Tokenizer roulette

[tiktokenizer.vercel.app](#) · [platform.openai.com/tokenizer](#) · [01_token_economics.ipynb](#)

Run sheet: DEMOS.md · Demo 1



Three token types, three different prices

The billing model matured: it is no longer just input and output

INPUT · PREFILL

1×

Your whole prompt, processed in **parallel** across tensor cores.

Compute-bound, near-peak hardware utilization — so throughput is high and unit cost is low.

The only cacheable category: the KV state can be reused.

OUTPUT · DECODE

5–6×

Generated **sequentially**, one full forward pass per token.

Memory-bandwidth-bound — utilization collapses, so each token costs far more to produce.

Never cacheable. It is new every single time.

REASONING · HIDDEN

5–6×

Internal thinking tokens on reasoning models — scratchpad work you never see in the response.

Billed at full output rates and the volume is invisible in your prompt.

A reasoning-effort setting is a pricing decision disguised as a quality setting.

Verbosity is more expensive than intelligence. Cap output length, demand structured responses, and set reasoning effort deliberately — that is the fastest cut available on the expensive side of the ledger.

Prefill/decode economics are hardware physics, not vendor pricing strategy. Reasoning-token billing per OpenAI and Anthropic published pricing, September 2026.

DEMO

5 min · Module 1

Watch a token get predicted

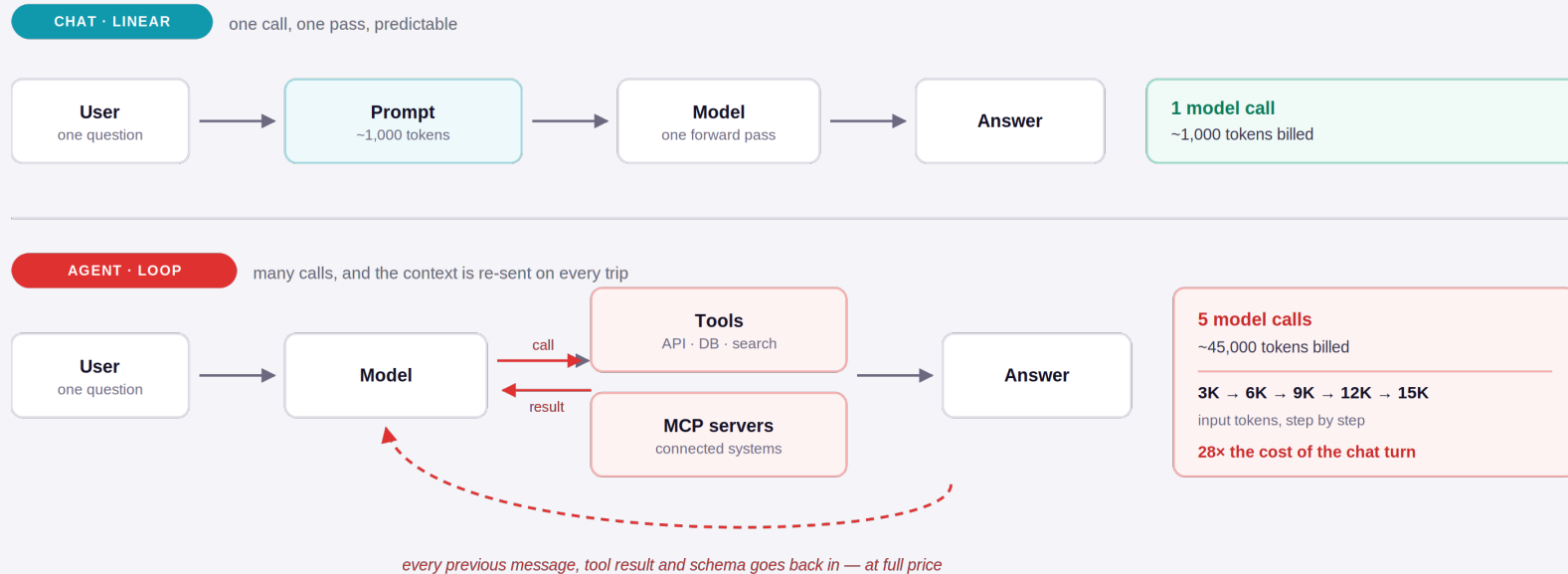
poloclub.github.io/transformer-explainer

Run sheet: DEMOS.md · Demo 2



Two shapes of consumption

The same question, asked of a chatbot and of an agent. Watch what happens to the input side.



A chatbot's cost is a line. An agent's cost is a loop. Every architecture decision in this course is really a decision about how many times the same context gets paid for – and whether you cache it

Three ratios that don't change when prices do

Memorise the ratios. Look the prices up on the morning you need them.

Generating a token costs

5–6× more

than reading one. Decode is sequential and memory-bound; prefill is parallel. Physics, not policy.

A cached input token costs

50-97.5% less

than a fresh one, on every major provider — if the prefix is stable.

Batch processing costs

50% less

flat, on every major provider, if you can tolerate the latency.

AND FOUR TIERS — THE VOCABULARY WE USE ALL AFTERNOON

FLOOR

\$0.14–0.30 / M in

VOLUME

\$0.75–1.00 / M in

NEAR-FRONTIER

\$2–5 / M in

FRONTIER

\$10–20 / M in

SAME SUPPORT ASSISTANT · 1M REQUESTS · 800 IN / 200 OUT



Same product behaviour, same prompt, same volume — **\$168 against \$18,000 a month**. Model selection is the largest single line on the bill, and the cheapest decision to change.

Computed from published provider rate cards, September 2026. Full rate card in the appendix. Prices move monthly — the ratios and the tier structure do not.

Long context has two pricing shapes. Know which one you are on.

Cross a vendor's long-context threshold and your unit price can double — silently, on every call, forever

TIERED — YOU PAY A CLIFF

GPT-6 Astra: \$10 → \$20 input, \$50 → \$75 output past the threshold

GPT-5.6 Terra: \$2 → \$4 input, \$12 → \$18 output

Gemini 3.1 Pro: \$2 → \$4 input, \$12 → \$18 output above 200K tokens

Effect: Cross the threshold and the whole request is billed at the higher rate — not only the excess tokens. The threshold is evaluated per request; stay below it next time and standard rates apply

FLAT — NO PREMIUM

Anthropic's current generation — Opus 5, Sonnet 5, Fable 5 — bills the full 1M-token window at standard rates

No surcharge above 200K; caching and batch discounts apply across the whole window

Google's current **Flash** and **Flash-Lite** text models list a single standard input/output rate across their available context windows; unlike Gemini Pro

Effect: Long-context workloads have materially different cost curves by **model family**, not merely by vendor.

Data residency (inference_geo) applies a 1.1× multiplier to all token categories — budget for it

"Just paste the whole export" is a pricing decision. On a tiered vendor it can double what you pay per token; on a flat vendor it cannot.

A widely circulated research summary claiming a >200K premium on Claude Sonnet is out of date — it described the 4.x generation. Current 1M-window models bill flat.

Prompt caching pays for itself on the second call

The provider stores the computed state of your prompt's stable prefix. You then rent it back at a tenth of the price.

ONE PROMPT, TWO HALVES

System prompt	3,000
Tool schemas	1,800
Policy & reference	700
CACHE BREAKPOINT 5,500 stable tokens above	
Retrieved context	2,000
Conversation history	1,200
User message	120

Everything above the line is identical on every call.
Everything below changes. Put it below, or you cache nothing.

WHAT FOUR CALLS COST

Call 1	WRITE 1.25×		\$0.0089
Call 2	READ 0.1×		\$0.0016
Call 3	READ 0.1×		\$0.0016
Call 4	READ 0.1×		\$0.0016

Break-even lands on call 2.

You pay a 25% premium once, then 90% off for as long as the prefix holds.

No cache, 4 calls:	\$0.0285	
With cache, 4 calls:	\$0.0137	52% less

Caching is not a feature flag — it is a **prompt-ordering discipline**. Put everything that never changes at the top, set the breakpoint, and stop paying twice for it.

Derived from Anthropic, OpenAI and Google published cache pricing, September 2026. Costs shown at near-frontier rates.

The forecast, worked end to end

One real support assistant, from the base equation to the number you would actually defend

Base equation

1M requests × (800 in + 200 out), volume tier

\$1,350

× M_agent = 28

the same question, answered by a 5-step agent

\$37,800

× M_context = 1.0

flat-priced vendor, no long-context cliff

\$37,800

× (1 + retry) = 1.10

failed tool calls and validation retries

\$41,580

+ invisible tail ~12%

embeddings, reranking, observability, review

\$46,570

The base equation was 3% of the answer.

Everything that mattered was a multiplier.

So forecast a range, not a number:

best \$12K · base \$47K · worst \$140K

Never forecast per token. Forecast per completed task, then multiply by task volume — and publish a range, because the multipliers are where the uncertainty lives.

Where an agent's money actually goes

One user question on a near-frontier model: chat versus a five-step agent that re-reads its context

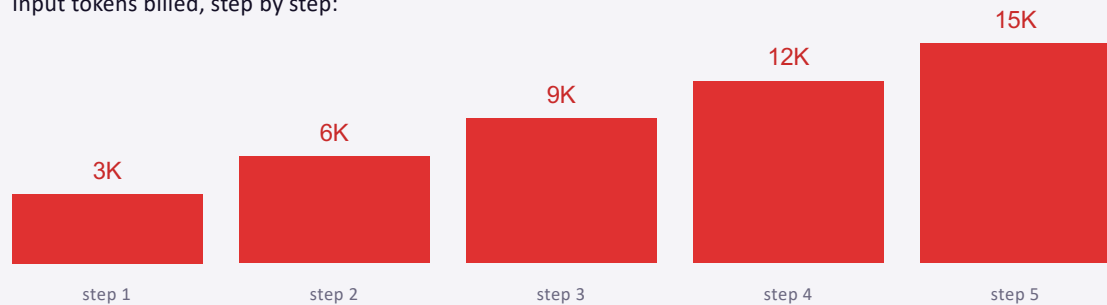
28×

the cost of the same question, answered by an agent

62%

of total agent inference bills is context the model has already read before.

Input tokens billed, step by step:



45,000 input tokens for one user-visible question.

Output barely moves; input explodes. Turn on prompt caching for the stable 80% and you recover roughly two thirds — and you are still 10× the chat cost. That is the 28× multiplier, derived rather than quoted.

Agent cost is input cost. Which means context engineering — not output tuning — is where agent bills are won. That is Module 2.

Computed at near-frontier rates (\$2 in / \$10 out / \$0.20 cache read). Stanford Digital Economy Lab (2025); Gartner (March 2026).

DEMO

8 min · Module 1

Build the forecast model live

01_token_economics.ipynb · artificialanalysis.ai · pricepertoken.com

Run sheet: DEMOS.md · Demo 3



WORKED EXAMPLE

5 min · ask the room, then advance

The CFO question

WHAT YOU KNOW

You are pitching a customer support assistant.

1,000,000 chats per month

800 input tokens per call — system prompt, tool schemas, retrieved context

200 output tokens per call

No caching enabled yet

Running on a volume-tier model at \$0.75 in / \$3.75 out

THE QUESTION

"What happens if adoption grows 5× and we go agentic next quarter?"

Before you advance: ask the room for a number. Most people multiply by five and stop there.

Take two answers from the room before you advance. **The wrong answer is the teaching moment.**

The CFO question, answered

The base equation is the easy part. The multipliers are the answer.

TODAY

\$1,350

1M chats × (800 in + 200 out) on the volume tier. This is the number in the business case.

× 5 ADOPTION

\$6,750

The intuitive answer, and the one most rooms give. It is not wrong — it is incomplete.

× 28 AGENTIC

\$189,000

Five tool-calling steps, each re-reading the accumulated context. Same feature. Same users.

WITH CACHING + ROUTING

\$71,000

Prefix caching on the stable 80%, output capped, routine intents moved down-tier. Still 53× today.

Adoption was never the risk. The architecture change was.

The honest answer to your CFO is not a number, it is a range with a named trigger: "\$7K if it stays conversational, \$70K–190K the quarter it becomes agentic — and here is the cap I want in place before that happens."

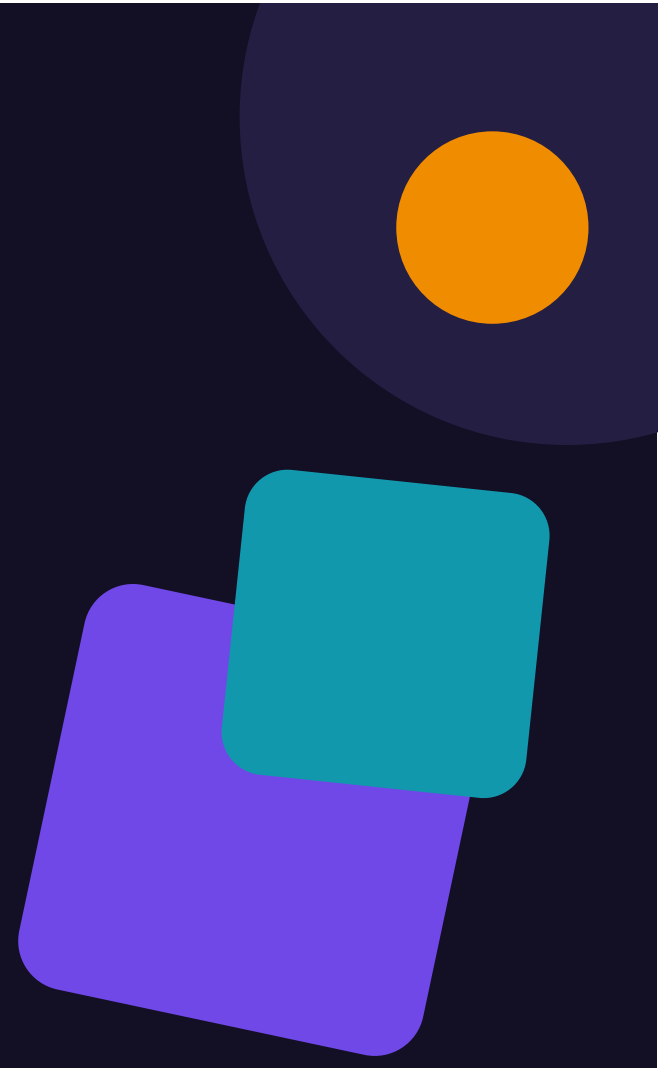
Computed at volume-tier rates, September 2026, using the 28× agentic multiplier derived earlier in this module.

MODULE 2

You are paying for the shape of your context

First, the anatomy of a real RAG and agent call. Then where the tokens accumulate, and the levers that reduce them without sacrificing quality.

50 MINUTES · 2 LIVE DEMOS · 1 WORKED EXAMPLE



UNDERSTAND

Where the tokens actually are

An illustrative enterprise RAG / agent call — 8,600 input tokens, component by component



76%

Tool schemas and retrieved context together account for roughly three-quarters of the input bill in production RAG and agent calls

The system prompt is only 5% in this example.

Optimizing it helps—but the larger opportunities may sit in retrieval, tool definitions, and repeated context.

In many tool-calling implementations, every enabled tool definition is included in the model context—even when only a few tools are relevant to the current task.

Audit before you optimise. Print the token count of every component of one real production call before you change a single word.

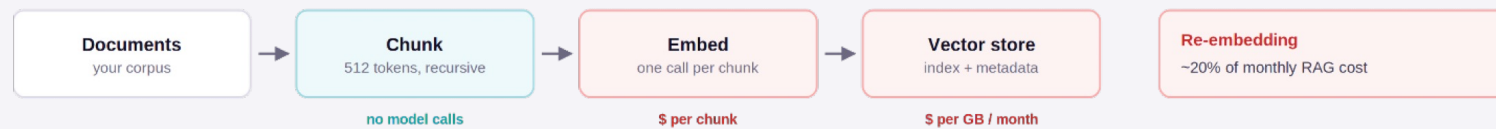
Representative enterprise RAG composition; consistent with published agent-trajectory breakdowns

UNDERSTAND

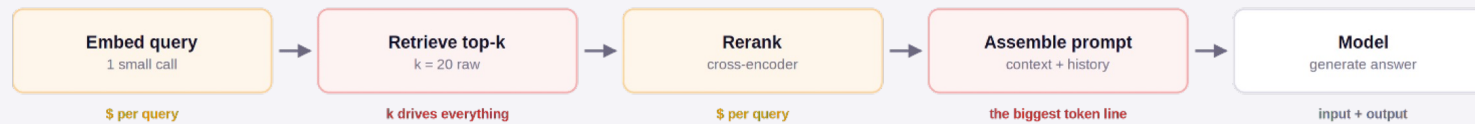
What a RAG stack actually charges you for

Two clocks run: one at build time, one on every single question

BUILD TIME · RUNS ONCE, THEN AGAIN ON EVERY DOCUMENT CHANGE



QUERY TIME · RUNS ON EVERY SINGLE QUESTION



Four costs live outside the model invoice

Embedding at ingest · re-embedding on every document change · vector storage per month · reranking per query
Teams forecast the model bill and get surprised by the system bill. Data cleaning alone runs 30–50% of a RAG project's cost.

k is the largest per-query token line.

Example: reducing retrieval from $20 \times 1,000$ -token chunks to 5×512 -token reranked chunks cuts retrieved context from 20,000 to 2,560 tokens. Whether quality holds must be proven on your eval set.

Only the last box is your generation-model inference cost. Everything to the left can create additional ingestion, retrieval, storage, embedding, and reranking costs that need to be forecast separately.

UNDERSTAND

The RAG costs your token dashboard does not show

Retrieval infrastructure is not free, and none of it shows up where you are looking

Embedding at ingest

ONE-TIME

One embedding call per chunk across the whole corpus. Cheap per call, large in aggregate — and easy to underestimate by an order of magnitude on a big document set.

Re-embedding

RECURRING

Every document change re-embeds its chunks. Change the chunking strategy or the embedding model and you re-embed everything.

Vector storage

RECURRING

Index + metadata create recurring storage and serving costs. Without lifecycle controls, stale vectors, duplicate versions, and oversized embeddings can quietly increase the footprint.

Reranking

PER QUERY

A reranker scores retrieved candidates before prompt assembly. It adds per-query cost and latency, but may let you send fewer—and more relevant—chunks to the expensive generation model.

Data preparation can also dominate implementation effort. Some 2026 industry estimates place cleaning and preprocessing at roughly 30–50% of RAG project build cost—but this varies heavily with corpus quality and should not be confused with monthly inference cost.

Put every one of these on the same forecast as the model call. A cost model that stops at the API is not a cost model.

Cockroach Labs (2026); industry RAG cost analyses, 2026.

“

Find the smallest possible set of high-signal tokens that maximize the likelihood of some desired outcome.

Anthropic, Effective Context Engineering for AI Agents — context as a finite attention budget, not a container to fill

This one sentence is the whole module. It is simultaneously the cost objective and the quality objective.

UNDERSTAND

A bigger context window does not fix this

Context rot: output quality degrades as input grows — before you reach any limit

18 models tested

frontier models tested got measurably worse as input length increased. Not some. Not most. All of them.

Context overflow is when input exceeds the model's limit. Context rot happens earlier — while there is still room, but the working context has become too large, noisy or badly organised to support reliable decisions.

Which is why every lever in the rest of this module is a cost lever and a quality lever at the same time:

- **More context does not guarantee more accuracy**

Past a point it is measurably less. The trade-off curve bends down, and most teams never look for the bend.

- **So "paste everything" can fail twice**

It costs more per call and it degrades the answer. There is no version of this where you are buying quality with money.

- **Good context reduction can improve both sides**

Compaction, selective retrieval and isolation can reduce cost while also removing distractors—but quality should still be validated with evals.

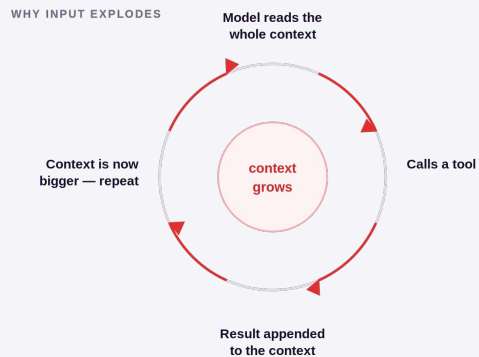
The cost objective and the quality objective point the same way. That is unusual, and it is the strongest argument you have for doing this work.

Chroma, "Context Rot" (2025), 18 frontier models; Anthropic, *Effective Context Engineering for AI Agents* (2026).

UNDERSTAND

Four techniques for controlling context growth

Reduce repeated, irrelevant, and accumulated context before it reaches the model again.



62% of total agent inference bills is context the model has already read before

Stanford Digital Economy Lab, Agentic AI Cost Attribution (2025)

Compaction

Summarize history as you approach the limit. Keep decisions, constraints and open items; discard redundant tool output.

Structured note-taking

Let the agent persist memory outside the window — a NOTES file, a to-do list. Persist durable state outside the active context and retrieve only the parts needed for the next step.

Just-in-time retrieval

Pass identifiers — paths, URLs, query handles — and load content at runtime instead of pre-loading the corpus into every call.

Sub-agent isolation

Delegate to agents with clean context windows; return only the result or compact summary the coordinator needs.

None of these make the model cheaper. **They make you stop buying the same tokens twice.**

Two ways to compress a prompt

One takes an hour and no tooling. The other is research-grade and situational.

BY HAND — DO THIS FIRST

- 01 **Dedupe repeated constraints.** 5–15% of a mature prompt is one rule stated three times by three authors.
- 02 **Convert verbose prose to concise structure where appropriate.** Lists and compact sections are often shorter and can make constraints easier to follow.
- 03 **Keep the smallest few-shot set that clears the quality bar.** More examples add recurring input cost and do not necessarily improve accuracy.
- 04 **Delete stale instructions.** They cost tokens and degrade output — free twice.
- 05 **Use compact delimiters.** ### instead of ===== SECTION =====.

Expect 20–40% off, with no measurable quality loss.

ALGORITHMIC — SITUATIONAL

Selective Context

prunes low-self-entropy units

~2× compression, minimal loss

LLMLingua

coarse-to-fine perplexity pruning

up to 20× compression, <2% quality loss

LLMLingua-2

token classification, no scoring pass

2–5× compression, up to 2.9× faster

LongLLMLingua

question-aware, contrastive to the query

~4× compression, +21.4% accuracy

LongLLMLingua made the answers **better**, not just cheaper — removing distractors is a quality intervention too. But every compression claim still needs a **fixed evaluation set** behind it.

SHRINK WHAT YOU SEND

Compress this. Never compress that.

The boundary matters more than the technique — this is where compression projects fail

SAFE TO COMPRESS

Verbose instructions, retrieved prose, background documents, conversation history, transcripts, policy text.

High redundancy; meaning survives paraphrase; no exact-token dependency.

"Would a careful human summary preserve everything that matters here?"

4,000 TOKENS OF POLICY PROSE → 1,200 TOKENS

COMPRESS, THEN RE-EVALUATE

Few-shot examples, tool descriptions, output specifications, reasoning chains.

Compressible, but the format itself carries meaning. Always re-run the eval set afterwards.

"Does the model rely on the exact structure here, or only on the content?"

8 FEW-SHOT EXAMPLES → 2 PLUS AN EXPLICIT RUBRIC

NEVER COMPRESS

Code, numbers, invoices, identifiers, dates, legal citations, contractual language — anything you assert as fact.

A dropped digit is not a quality regression. It is an incident.

"If one token were silently altered here, would anyone notice before the customer did?"

AN INVOICE LINE ITEM, A CASE ID, A CONTRACT CLAUSE

Compression is a **per-component** decision, never a per-request one. Split the prompt, compress the safe parts, leave the rest byte-exact.

SHRINK WHAT YOU SEND

Chunking is a cost decision, not just a quality decision

What recent benchmarks tell us: complexity does not guarantee better RAG

Strategy	End-to-end accuracy	Retrieval recall	Model calls at ingest	Verdict
Recursive character splitting	69% — best	Strong	Zero	Start here
Fixed-size (512 tokens)	67%	Strong	Zero	Fine fallback
Page-level	64.8%	Moderate	Zero	Paginated financial PDFs
Semantic chunking	54%	91.9% — highest	Embedding calls	Wins the wrong metric
LLM-based / agentic	Varies	Varies	1 call per chunk	High-value corpora only

Semantic chunking lost the answer

Its fragments averaged 43 tokens. Recursive splitting, with zero model calls, beat every more expensive alternative.

It is also ~14× slower

≈0.33 MB/s versus 4.82 MB/s for token-based. On a large corpus that is hours instead of minutes, plus embedding spend.

Overlap is a tunable, not a default

A January 2026 systematic analysis found overlap added no measurable benefit in its setup while raising indexing cost.

FloTorch 2026 (50 papers, 905,746 tokens); Chonkie throughput benchmarks; Weaviate reports up to a 9% recall gap between best and worst chunking on the same corpus. Best starting config: 512 tokens, 50–100 token overlap.

The token tax of being right

972 answers · 3 machines · expert-validated manufacturing-safety benchmark

26×

the per-query token cost, for +7.7 accuracy points

Long-context prompting



Semantic RAG



Long context won — and cost 26 times as much per query.

7.7 points is cheap insurance in a safety-critical workflow and indefensible in a routine internal FAQ.
That is not an architecture preference; it is a business decision with a number attached.

Frame it as an **accuracy–cost frontier**, and require every use case to state where on that frontier it sits, and why.

"The Token Tax of Epistemic Accuracy," arXiv:2606.20898 (2026).

STOP RE-SENDING

Three different things people call "caching"

They intercept different traffic and fail differently — stack all of them

L1

Exact-match cache

<1 ms

Byte-identical query returns a stored response. Redis or an in-memory dict. Saves 100% of the call. Modest hit rate alone, but nearly free and impossible to get wrong.

L2

Semantic cache

3–10 ms

Embeds the query and returns a stored answer above a similarity threshold (typically 0.88–0.95). Saves 100% of the call on a hit; published hit rates of 61.6–68.8%. This layer has a real failure mode.

L3

Prompt / prefix cache

provider-side

The provider reuses computed KV state for an identical prefix. 90–98% off cached input. Does not skip the call — it makes the input portion nearly free. Requires prefix stability.

L4

The model call

500–2,000 ms

Full price, full latency. Everything above exists to reduce how often you land here. Instrument the hit rate at every layer separately.

L1 and L2 skip the call entirely; L3 only discounts the input. **They are complements, not alternatives** — and teams routinely implement one and claim the benefits of all three.

GPT Semantic Cache, arXiv:2411.05276. Provider cache-read discounts per Anthropic, OpenAI, Google and DeepSeek documentation, September 2026.

STOP RE-SENDING

Put the stable part first, or you pay full price on every call

L3 is the layer with a prerequisite, and this is it

CACHE MISS/REBUILD

Session timestamp injected at the top of the system prompt

User ID or tenant name interpolated into the opening instruction

Tool schemas re-serialised in non-deterministic key order

Retrieved chunks placed before the static policy block

Dynamic content changes too early in the prefix, so reusable state cannot be matched. The provider processes those input tokens again at the applicable uncached/write rate.

CACHE HIT

Static content first, fixed order: system prompt → tool schemas → stable policy and reference material

Then the dynamic tail: retrieved context → history → user message

Deterministic serialisation for anything machine-generated

Cache breakpoint set at the end of the stable block

Stable content comes first; dynamic content follows. Repeated calls can reuse the stable prefix at the provider's cached-input rate.

The lesson is universal; the multiplier is not. Prefix order determines whether caching can work. The exact discount and write economics depend on the provider and model.

DEMO

8 min · Module 2

Cache hit, cache miss, invalidation trap

02_prompt_caching.ipynb · live API key · prints a running cost ledger

Run sheet: DEMOS.md · Demo 4



STOP RE-SENDING

The failure mode nobody demos

L2 is the only layer that can return a confidently wrong answer — and the failure is silent.

THE FAILURE

"Cancel the order."

versus

"Cancel the standing order."

Cosine similarity is very high. The correct actions are opposite — and one is irreversible.

"What is the cost of returning a confidently wrong cached answer here?"

HIGH-RISK FOR SEMANTIC RESPONSE CACHING

Personalized, account-specific, legal, medical, financial, transactional, or permission-sensitive answers require stricter isolation, freshness controls, and often should bypass semantic caching entirely.

HOW TO CONTAIN IT

Use stricter thresholds—or bypass semantic caching—for high-risk intents.

Namespace the cache per tenant, per user, per entitlement level.

Set a TTL that matches how fast the underlying truth changes.

Log every hit with its similarity score, so you can audit afterwards.

"Could this cached answer leak across a tenant or a permission boundary?"

ISOLATE BY TENANT · TTL BY DATA FRESHNESS
· THRESHOLD BY RISK CLASS

HOW TO REPORT IT HONESTLY

Hit rate alone rewards unsafe caching.

You can drive it to 90% by lowering the threshold and quietly serving wrong answers. The dashboard will look excellent.

Track hit rate and false-hit rate as a pair, and report them together, always.

"Do we sample and human-review cache hits, or do we only count them?"

REPORT: HIT RATE + FALSE-HIT RATE + SAMPLED REVIEW, SAME DASHBOARD

A cache hit rate reported without a false-hit rate is a **savings claim with the risk removed from the page**. Never accept one without the other.

STOP RE-SENDING

A 90% discount is not a 90% saving

Do this arithmetic out loud, before someone in the room does it for you

~30%

realistic total-bill reduction from prefix caching alone

Thirty percent is an excellent return for one hour of work. It is just not ninety.

The headline is real: cache reads are 90% off input tokens.

But caching touches only the input side — and output is 5–6× the price of input. At a realistic 80% prefix-cache hit rate on a typical workload, that 90% discount lands as roughly 30% off the total bill.

The same discipline applies to every number in this deck.

85% routing savings is a benchmark result on a wide model spread. A calibrated cascade on a real production workload returned ~30%. Budget for the second number and celebrate the first.

Quote savings against **the total bill**, never against the line item you optimised. That is the difference between a FinOps report and a vendor pitch.

Derived from published cache-read discounts and the 5–6× output/input price ratio; consistent with reported production outcomes of 40–80% on cacheable traffic.

DEMO

10 min · Module 2

Cut the bill, prove the quality held

[03_context_compression.ipynb](#) · [04_rag_cost.ipynb](#) · [05_semantic_cache.ipynb](#)

Run sheet: DEMOS.md · Demo 5



WORKED EXAMPLE

5 min · ask the room, then advance

Prompt surgery

WHAT YOU KNOW

A real enterprise prompt stack — 8,600 input tokens per call, 400,000 calls a month, near-frontier model.

500-token system prompt with three duplicated policy statements

2,500 tokens of tool schemas — two tools nobody has called in six months

4,000 tokens of retrieved context, top-20 chunks, unranked

1,500 tokens of full verbatim conversation history

A session timestamp on line two. No output length limit.

THE QUESTION

Which single change saves the most?

Most rooms say "compress the system prompt." It is the smallest line on the slide.

Take two answers from the room before you advance. **The wrong answer is the teaching moment.**

Prompt surgery, answered

Four changes, in descending order of what they actually return

DELETE 2 DEAD TOOLS

−1,200

Tokens per call, for zero risk and about ten minutes of work. Nobody has called them since March.

RERANK, THEN $k=5 \times 512$

−1,440

Top-20 raw becomes five reranked chunks. Groundedness holds; the eval set proves it.

MOVE THE TIMESTAMP

−0

Saves no tokens at all — and unlocks a cache read on 5,500 of them, on every call from the second onward.

TRIM PROMPT + HISTORY

−950

The work everyone starts with. Real, worth doing, and the smallest of the four.

8,600 → 5,010 tokens, and 5,500 of what remains is now cacheable.

Roughly \$6,900 a month becomes roughly \$1,600 — a 77% cut with no product change. The largest single line was two tool schemas nobody remembered registering, and the highest-leverage change saved no tokens at all.

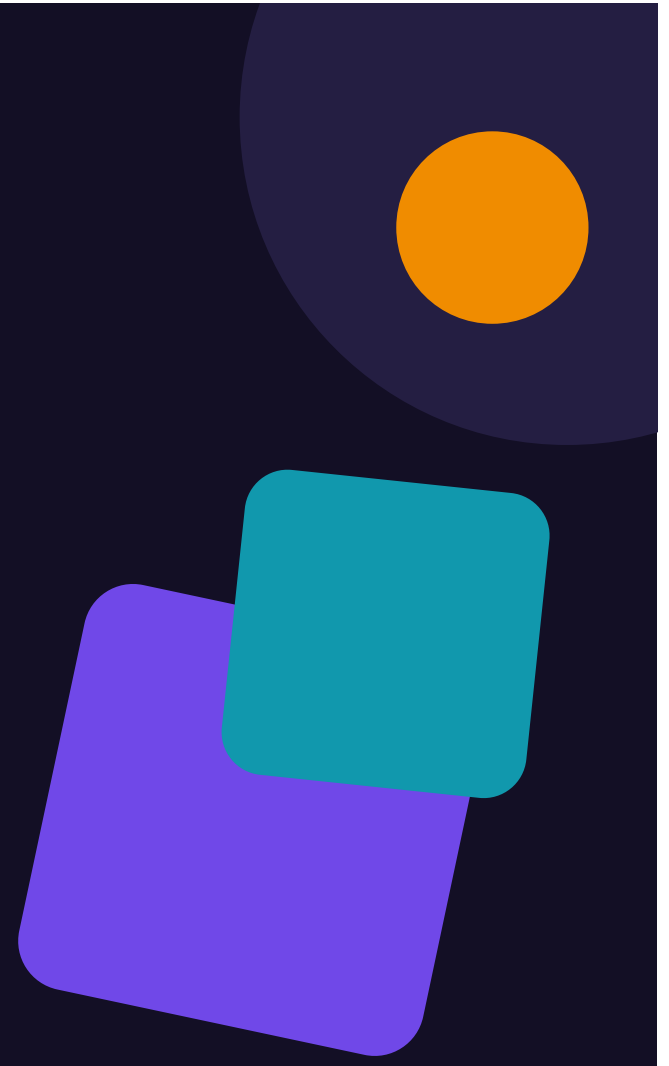
Computed at near-frontier rates, September 2026. Run the fixed eval set before and after — a saving you cannot prove is a regression you have not found yet.

MODULE 3

The biggest line on your bill is a dropdown

Model tiers, what routing actually is, what it actually returns, and the unit that decides whether a cheaper model was cheaper.

50 MINUTES · 1 LIVE BUILD · 1 REFRAME



Six months later, the frontier model is answering "is this email urgent?"

How every over-provisioned AI stack gets built — one reasonable decision at a time

Week 1

A team picks the best available model for their first AI feature. It works.

Month 2

They ship three more features on the same model. Nobody stops to ask whether every query needs it.

Month 6

80% of the inference budget is going to a frontier model doing classification, summarisation and yes/no triage.

Month 7

The invoice arrives. The architecture is already built, and the choice is no longer cheap to make.

You are not paying for better answers. You are paying for capability you are not using.

Teams that pull thirty days of calls and sort them by real complexity typically find 60–70% of their token volume routed to an overqualified model. The audit takes about a week. The savings are permanent.

\$40K → \$24K

monthly API spend for one team that audited usage and routed simpler subtasks to cheaper models — no product change.

The discipline of knowing **when not to call the model** is becoming as important as knowing how to prompt it.

Analysis of 2.4B enterprise API calls (2026); practitioner reports collected in industry coverage, 2026.

Four tiers, not two vendors

Stop asking "which model is best" and start asking "what is the cheapest tier that clears the bar"

\$

Floor

\$0.14–\$0.30 in

Classification, entity extraction, intent routing, reformatting, tool-argument shaping, deterministic transforms.

If an eval can score it automatically, it probably belongs here.

\$\$

Volume

\$0.75–\$1.00 in

Summarisation, structured processing, routine grounded Q&A, first-draft generation.

The workhorse tier. It should carry most production traffic, and rarely does.

\$\$\$

Near-frontier

\$2–\$5 in

Multi-step reasoning, code generation, grounded analysis, ambiguous customer-facing responses.

The current default for most teams — which is precisely the problem.

\$\$\$\$

Frontier

\$10–\$20 in

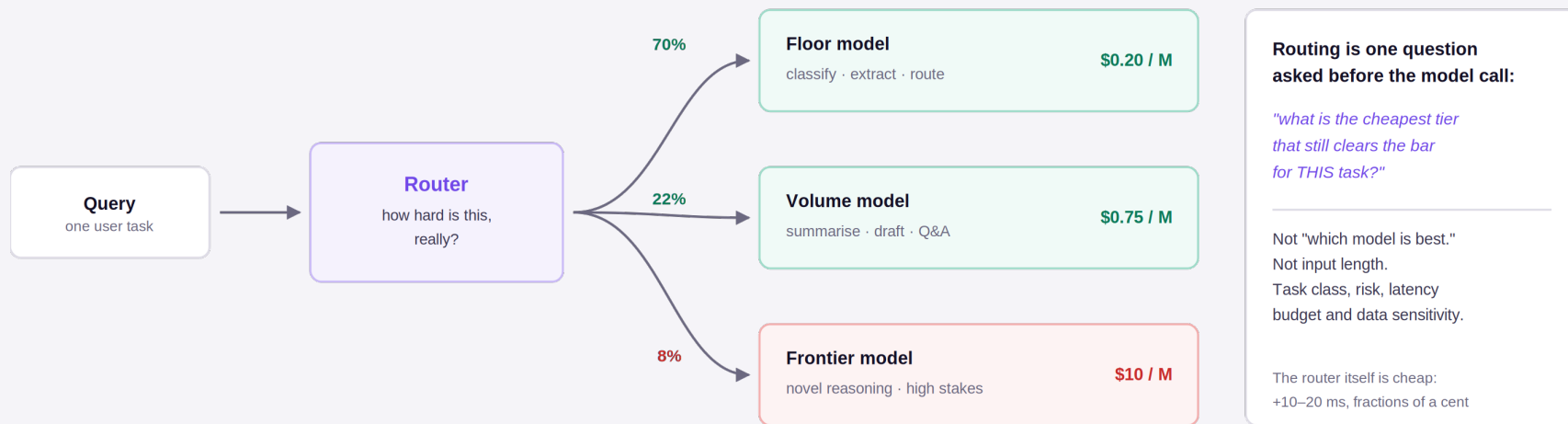
Novel reasoning, high-stakes synthesis, long unattended agent runs where an error is expensive to clean up.

Justified far less often than it is used — but genuinely justified sometimes.

Route on **task class, risk, latency budget and data sensitivity** — never on input length alone, which is the naive implementation everyone builds first.

What routing actually is

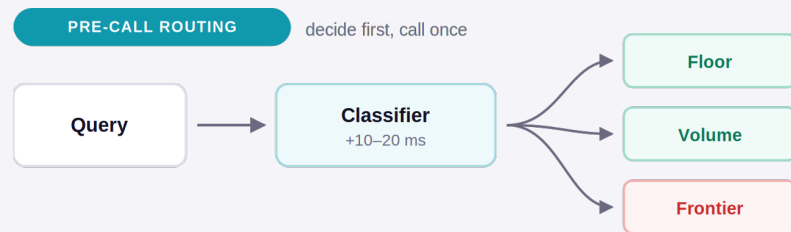
One cheap question, asked before the expensive one



The router is not a model upgrade. It is a dispatcher — and the traffic mix it produces is the number your finance team should be asking you for every month.

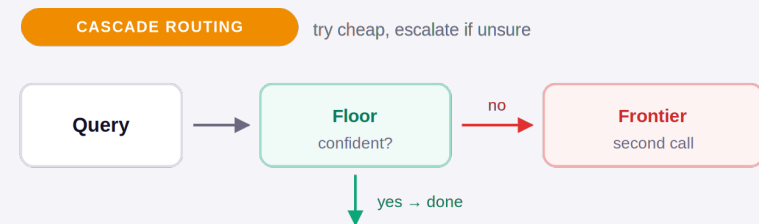
Two ways to build it

Decide before the call, or try cheap first and escalate



One model call. Latency stays flat.

Fails when query types drift from the training data.



Some queries pay for two calls.

Fails when the confidence score is miscalibrated — which is the whole engineering problem.

Most production systems end up hybrid: pre-route the task types you recognise, cascade the ambiguous 10–20%, and give the routing policy an owner.

A third pattern is spreading in coding workflows: a cheap model runs the loop and escalates, or an expensive one runs the loop and delegates the routine steps.

Both fail the same way in the end: **nobody owns the routing policy**. Give it an owner and an eval harness in the same pull request as the router itself.

Cursor Router, OpenRouter AutoRouter, Databricks Unity AI Gateway Smart Routing, Claude's Advisor tool, Cognition's Devin Fusion (2026).

The number to budget with, and the number to celebrate

The gap between the top rows and the bottom rows is the most important thing in this module

Source	What it is	Reported result	Setting
FrugalGPT (Stanford, 2023)	The origin paper for model cascades	50–98% cost reduction at matched quality	Benchmark
RouteLLM (LMSYS / Berkeley)	A learned routing framework — the routing brain, not a gateway	~85% cost cut at 95% of GPT-4 quality; 14–26% routed to the strong model	Benchmark (MT-Bench)
RouterArena (ICLR 2026)	The open leaderboard — standardised datasets, metrics, live results	The current way to compare routers on like-for-like terms	Benchmark platform
OrcaRouter (May 2026)	Contextual bandit that adapts from deployment feedback	75.5% accuracy at \$1.00 per 1,000 queries; #2 on RouterArena	PRODUCTION-ORIENTED
Databricks Smart Router (2026)	Request-level routing inside an internal AI gateway	>30% average task cost reduction at matched quality	PRODUCTION — internal
UCCI (2026)	Calibrated token-level uncertainty sets the escalation threshold	31% cost reduction (95% CI 27–35%) at micro-F1 = 0.91	PRODUCTION — 75,000 queries

Benchmarks with wide model spreads produce 85–98%. Real production workloads produce **about 30%, with a confidence interval**. Budget for the second number and celebrate the first.

FrugalGPT (Stanford, 2023); RouteLLM (LMSYS/Berkeley/Anyscale); RouterArena, ICLR 2026; OrcaRouter, arXiv:2605.30736; UCCI, arXiv:2605.18796; Databricks, Managing AI Coding Costs at Scale (2026).

What you would actually deploy in 2026

A router decides which model. A gateway is where you enforce budgets, keys and observability. Most teams need both, and they are not the same product.

LiteLLM

OPEN SOURCE · SELF-HOSTED

OpenAI-compatible proxy across 100+ providers. Virtual keys, spend tracking, budgets, caching. The default when you want control and your own infrastructure.

OpenRouter

HOSTED · ZERO-OPS

One key and one endpoint in front of 500+ models, with consolidated billing. The fastest way to stop managing provider accounts.

Portkey

OPEN CORE + ENTERPRISE

Conditional routing, fallbacks, semantic caching and guardrails, with observability treated as a first-class feature rather than an add-on.

Weave Router

CODING HARNESSES

Per-action model selection for Claude Code, Codex and Cursor, scored against code quality and token cost.

Inworld Router

REAL-TIME / VOICE

Conditional routing and traffic splitting for latency-critical applications, with speech built in.

RouteLLM

RESEARCH FRAMEWORK

Not a gateway. A framework for training and evaluating learned routers — the brain, not the control plane.

Two cautions. Vendor-published savings figures are marketing until you reproduce them on your own traffic — and adding a gateway adds a dependency in the request path, so it needs the same reliability review as any other.

Product capabilities verified September 2026. Routing research and production results are on the previous slide; the two are not interchangeable.

DEMO

15 min · Module 3

Build a three-tier router live

06_routing_cascade.ipynb · live keys for at least two tiers · includes the calibration step and the quality gate

Run sheet: DEMOS.md · Demo 6



Cost per token is the wrong unit

You do not buy tokens. You buy completed tasks — and you pay for the ones that fail too.



divide by the success rate — always

Track it per model, per route and per agent, with a hard ceiling per task. It is the only unit that survives a CFO review of an agentic system, and the only one that tells you whether a cheaper route was actually cheaper.

Three things go into it, and everyone forgets the third.

What the attempt cost — billed tokens across the entire trajectory. Every step, every tool call, every retry. Not one API call.

How often it worked — the probability the task actually completed correctly. A 70% success rate means you paid for 100 attempts and got 70 outcomes.

What the failures cost you — when a wrong answer escapes into a business process and a human has to unwind it. This is the term nobody puts in the spreadsheet.

Reliability is a **cost lever**, not only a quality attribute. This is the sentence that changes how a CFO reads a model comparison.

Formalisation per DoIT, "Cost per task, not cost per token" (August 2026).

The same comparison, run three ways

Sticker price says one thing. Cost per solved task says another. Cleanup cost settles it.

MODEL A · THE CHEAP ONE

Price per attempt	\$0.40
100 tasks attempted	\$40.00
Success rate	70%
Tasks actually solved	70

\$0.57 per solved task, before cleanup

Wrong answers that escaped	8
Cleanup at \$60 each	\$480

\$7.43 true cost per solved task

MODEL B · 3× THE STICKER PRICE

Price per attempt	\$1.20
100 tasks attempted	\$120.00
Success rate	94%
Tasks actually solved	94

\$1.28 per solved task, before cleanup

Wrong answers that escaped	2
Cleanup at \$60 each	\$120

\$2.55 true cost per solved task

vs

The model that is 3× the sticker price is **2.9× cheaper per solved task**. A cheaper model with a lower success rate and a higher leak rate is routinely the more expensive model.

Illustrative worked example using the DoIT cost-per-task framework. Cleanup cost is the modelled expense of a human unwinding one wrong answer.

And here it is on real benchmarks

Cost per successful task on published 2026 results — including the cost of cleaning up the failures

Workload	Model	Success rate	Cost per successful task	What flips the ranking
SWE-bench Verified	GPT-5.6 Sol	96.2%	~\$1.56 per solved issue	High success rate compounds
SWE-bench Verified	Claude Opus 4.8	88.6%	~\$3.06 per solved issue	7.6 points of success = 2× the cost
τ-bench (airline support)	Claude Opus 4.8	—	\$1.47 per correct task	4 policy violations · \$100 cleanup
τ-bench (airline support)	GPT-5.5	—	\$2.63 per correct task	9 policy violations · \$225 cleanup

Note the reversal between the two halves of the table.

On SWE-bench the model with the higher success rate wins on cost. On τ-bench the ranking flips again once cleanup is priced in — because cleanup cost frequently exceeds the API bill. The model that wins on tokens can lose badly on unattended, high-stakes work.

DoIT, "Cost per task vs cost per token" (August 2026). Cleanup costs are the modelled expense of unwinding policy violations.

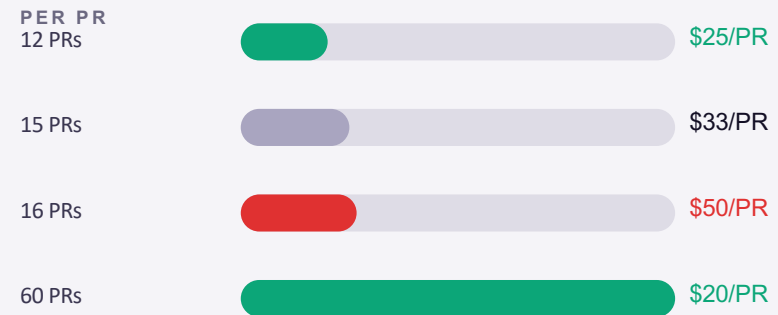
The engineer who spends the most is your cheapest engineer

Same team, same tools, one month of AI coding spend — ranked two different ways

WHAT THE SPEND DASHBOARD SHOWS



WHAT THE VALUE DASHBOARD SHOWS · PRs MERGED, AND COST



A \$500 cap would have throttled Engineer D — the one delivering work at \$20 per merged PR — and left Engineer C untouched at \$50.

Engineer D is not the problem. Engineer C is, and no spend dashboard in the room would have found them. Spending more is not waste when the output scales faster than the bill.

Shift from **token maxing** to **value maxing**. Every cost metric on your dashboard needs an outcome metric beside it, or you will optimise away your best performers.

Illustrative figures, scaled to Uber's reported range of \$150–\$250 per engineer per month typical and \$500–\$2,000 for power users (2026).

Few-shot, fine-tune, or distil

A volume-and-stability question, not an ideology question

FEW-SHOT PROMPTING

Under ~1,000 examples, or a task still changing week to week.

Zero upfront cost, but you pay the example tokens on every call — softened a lot when the block sits in a cacheable prefix.

"Is this task still changing faster than we could retrain?"

START HERE, ALWAYS. IT IS THE CHEAPEST WAY TO LEARN WHAT THE TASK ACTUALLY IS.

FINE-TUNING

Above ~5,000 high-quality examples, on a stable task, at sustained volume.

Buys a shorter prompt and lets a smaller base model match a larger one. Costs training, hosting and retraining.

"Has this task been stable for a quarter, and is the volume genuinely large?"

BREAK-EVEN IS ARITHMETIC: FEW-SHOT TOKENS × PRICE × VOLUME vs TRAINING + HOSTING

DISTILLATION

You have a strong model producing good outputs and want that behaviour at floor-tier prices.

Use the large model to label data, then fine-tune a small one on those labels. The 2026 default for high-volume narrow tasks.

"Could we generate our training set from the expensive model we already run?"

FRONTIER LABELS → SMALL MODEL → FLOOR-TIER INFERENCE COST

Compute the break-even before you argue about it: **(few-shot tokens × price × monthly volume)** versus **(amortised training + hosting + per-call delta)**.

Self-host or buy: the honest math

All-in cost of a single H100, and what it has to displace to pay for itself

Line item	Monthly	Displacing	Break-even volume
Cloud GPU rental (1× H100)	~\$1,440	Floor tier (\$0.14/M)	~5.7B tokens/mo
DevOps / SRE loading	~\$1,500	Near-frontier (\$3/M)	~430M tokens/mo
Networking, storage, observability	~\$300	Frontier (\$5/M)	~256M tokens/mo
All-in	~\$3,240	At 60–70% sustained utilisation	Below ~250M tokens/mo: buy

Engineering overhead is a 3–5× multiplier on raw GPU rental.

Every self-hosting business case that ignores it is wrong, and most of them do. Deloitte's crossover analysis puts self-hosting ahead only above roughly 67–84 billion tokens a year — about 7 billion a month.

So the honest question is not "should we run our own models?" It is "do we have a workload predictable enough, and large enough, to keep a GPU busy?"

10×

your real cost per token at ~10% GPU utilisation — worse than premium API pricing

Measure utilisation, not cost per token.

Industry benchmark is 65–75%. Most enterprise deployments run 30–50%, and inference-only analyses put it at 15–30%. Business-hours traffic on a 24-hour asset is low utilisation by construction.

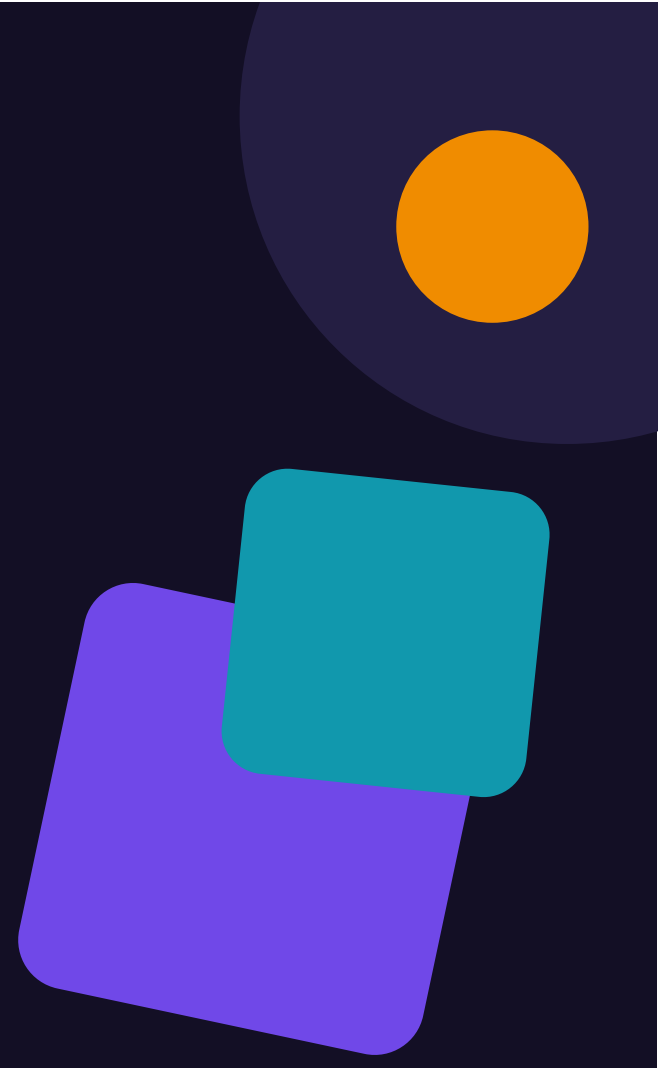
2026 deployment economics. Serving stack: vLLM for general throughput, SGLang for prefix-heavy RAG and multi-turn. GPU rental assumes a reserved or volume contract; on-demand H100 is typically \$2,000–\$3,000+. Deloitte (2026).

MODULE 4

You cannot govern what you cannot attribute

The FinOps loop for AI: track, attribute, control, optimize — plus the circuit breakers that make an uncertain forecast survivable.

42 MINUTES · 1 LIVE DEMO · 1 WORKED EXAMPLE



This stopped being a side quest

FinOps Foundation, State of FinOps 2026 — 1,192 practitioners representing \$83B+ in annual cloud spend

98%

of FinOps practitioners now manage AI spend — 63% in 2025, 31% in 2024. FinOps for AI is the #1 forward-looking priority.

78%

of FinOps teams now report to the CTO or CIO, up 18 points since 2023. Only 8% report to the CFO — this is an architecture capability, not a finance one.

#1

skill gap: AI value management. The three named obstacles are visibility into AI cost, allocating it to business units, and determining ROI.

The Linux Foundation launched the Tokenomics Foundation in June 2026 — modelled on how the FinOps Foundation standardised cloud cost discipline a decade earlier. Its stated purpose is open standards, benchmarks and best practices for the economics of AI infrastructure.

Many organisations report being asked to self-fund their AI investments through efficiency gains. That is this class, stated as a mandate.

Track → Attribute → Control → Optimize

Run it in this order. Skipping a step is why AI governance programmes stall.

1

TRACK

Capture usage from provider responses — that is the authoritative billing record, not your own estimate.

Normalise carefully: OpenAI counts cached tokens inside prompt_tokens; Anthropic reports cache reads in a separate bucket. Sum them naively and you double-count.

2

ATTRIBUTE

Propagate metadata on every single call: business unit, use case, environment, model, workload type, user, session.

Without this you have a total, not a cost model — and a total cannot be managed, only regretted.

3

CONTROL

Enforce at the gateway, never in application code.

Three-tier alerting: 50% notify and pause non-production, 80% require approval for new experiments, 100% hard stop on non-critical traffic. Plus per-session circuit breakers for agents.

4

OPTIMIZE

Now — and only now — sort attributed spend by feature and difficulty.

Move routine work down-tier, refactor for cache hit rate, trim retrieval. Everything from Modules 1 to 3 goes here, aimed by data rather than by intuition.

Most teams start at step 4 because it is the fun one. **Optimising without attribution means you cannot prove the saving, cannot find the next one, and cannot stop it regressing.**

A documented production incident in the LLM community: an agentic workflow generated \$47K in compute before a budget alert fired. The fix was a real-time enforcement layer. The incident happened because none existed.

The minimum viable tagging schema

Put this on every LLM call from day one. Retrofitting attribution is far more expensive than adding it.

WHO PAYS

business_unit — the chargeback target

cost_tier — 1 foundation / 2 enablement / 3 differentiation

use_case_id — stable per use case, never per deployment

user_id — per-user attribution and abuse detection

WHAT RAN

model_id — provider and exact model version

workload_type — inference / embedding / fine-tuning / training

environment — dev / staging / prod

session_id — the key to agent-loop detection and per-task costing

WHY IT MATTERS

Without **business_unit** you cannot do showback — and showback is what changes behaviour

Without **session_id** you can only compute cost per call, never cost per completed task

Without **environment** your dev experiments contaminate production unit economics

Without **cost_tier** you cannot decide what is centrally funded and what is charged back

Target 95%+ tag coverage within 30 days — and report the coverage number itself

Start with **showback, not chargeback**. Show teams their number for a quarter before you charge them for it — you get the behaviour change without the political fight.

Instrument to the standard, not to a vendor SDK

OpenTelemetry GenAI semantic conventions — but know exactly what you are adopting

Attribute / metric	What it carries	Note
<code>gen_ai.usage.input_tokens</code>	Input tokens per call	v1.27+ — older <code>prompt_tokens</code> still co-emits in many frameworks
<code>gen_ai.usage.output_tokens</code>	Generated tokens per call	Query both naming generations during the transition
<code>gen_ai.provider.name</code>	The model provider	Renamed from <code>gen_ai.system</code> in v1.37.0 (Aug 2025)
<code>gen_ai.request.model</code>	The exact model invoked	Essential for tier-mix reporting
<code>gen_ai.client.token.usage</code>	Token histogram, by token type	A metric, not an attribute
<code>invoke_agent</code> → <code>chat</code> → <code>execute_tool</code>	The span hierarchy for an agent run	This is what makes per-task costing possible

Status check before you standardise on it: **as of mid-2026 no GenAI-specific span, metric or attribute is marked Stable**. Adopt it as the direction of travel, and pin your version.

opentelemetry.io GenAI observability; semantic convention status verified 2026.

"We have a spend limit" is not a control

What a hard cap actually requires — and the prerequisite nobody mentions

1

Budget scopes

Global proxy · team · internal user · virtual key · per-model · end customer · tag and project.

Reset windows configurable in seconds, minutes, hours or days. Unset means never resets — a bug waiting to happen.

2

Rate limits

Tokens per minute, requests per minute, maximum parallel requests, per-model TPM and RPM ceilings.

Rate limits protect availability. Budgets protect the invoice. You need both, and they fail differently.

3

The prerequisite

Budgets are enforced against spend read from a database.

A gateway deployed without persistent state cannot enforce a budget at all.

Governance has an architecture prerequisite, and it is the thing most often skipped in a proof of concept.

Before you call it a control, answer three questions: **at what scope, read from where, and what happens on breach?**

LiteLLM proxy budget and rate-limit documentation. Comparable capabilities exist in Portkey, Helicone, Kong AI Gateway, AWS Bedrock and Azure.

Circuit breakers: three ceilings, any one of which fires

Enforce them at the gateway, never inside the model or the agent framework

STEP CEILING

Cap the maximum tool calls per task. Fifty is a reasonable default for most workflows.

Anything beyond that is not a hard problem — it is a loop, and it will not resolve itself.

"What is the most tool calls a legitimate task here has ever needed?"

TOKEN & SPEND CEILING

Cap total tokens per session (200K is common) and maximum spend per trace ID (say \$5.00).

Enforced against the session_id you tagged earlier — which is why the tag schema comes first.

"What is the most this single task could ever justify spending?"

WALL-CLOCK CEILING

Cap elapsed time per task. Five minutes is typical for interactive work.

Catches what token counting misses: slow external tools, retry storms, deadlock between agents.

"How long before a user would have given up anyway?"

HTTP 429 at the gateway, not a warning in a log file.

Uber's answer to burning a year's budget in four months was not a better forecast. It was a cap. Caps are how you make an uncertain forecast survivable — and they are the only control that works while everyone is asleep.

A circuit breaker is not a budget. A budget protects the month. A circuit breaker protects the next twenty minutes.

But halting is the blunt instrument. Steering works better.

Graduated friction keeps the work moving while the spend curve bends

THE LADDER, IN ORDER

01 **Visibility**

Near-instant feedback on ongoing spend, with specific tips on cheaper models. Every company that solved this started here.

02 **Spend gates**

Self-clearing warnings first; explicit budget approval through a management chain higher up.

03 **Downshifting**

Route the developer to a lower-cost model rather than suspending access. They keep working; the bill bends.

04 **Suspension**

The limit case, usually temporary — the start of a conversation, not a punishment.

A cost guard watches budget velocity and injects instructions to keep outputs succinct — instead of ending the run.

78%

lower average agent spend

67% → 96%

task completion, versus simple throttling

Coinbase halved its bill without a wall.

Better defaults, routing and caching in an internal gateway — and then the discovery that 91% of employees never hit their caps anyway. The caps were solving a problem 9% of the company had, while the defaults were solving one everybody had.

Halt when the run is broken. **Steer when the run is merely expensive.** Most organisations only ever build the first one.

TokenOps enforcement benchmark (2026); Databricks, Managing AI Coding Costs at Scale (2026); Coinbase, June 2026.

The metric that hides, the metric that surfaces

Same spend. Different dashboard. Completely different management decision.

HIDES PROBLEMS

Cost per token — falls steadily while your bill rises, because it measures the market, not you

Total monthly spend — moves too slowly to catch a runaway agent before month end

Cache hit rate alone — improves when you lower the threshold and serve wrong answers

Tokens processed — a vanity metric that rewards consumption

By the time any of these move, the incident already happened and you are explaining it, not preventing it.

SURFACES PROBLEMS

Cost per completed task, with a hard ceiling per task — the leading indicator for agents

Cost per business outcome — per resolved ticket, per qualified lead, per approved document

Tokens per task at P50 and P95 — P95 is where runaway loops hide; the median never shows them

Percentage of traffic on the economy tier — the health check for your routing policy

Forecast versus actual, weekly — the maturity signal a CFO recognises immediately

Report cost **alongside quality** on the same dashboard. A dashboard that shows dollars without task success rewards exactly the wrong optimisation.

DEMO

12 min · Module 4

A hard stop that actually fires

07_finops_telemetry.ipynb · optional LiteLLM proxy in Docker · langfuse.com free tier for the trace view

Run sheet: DEMOS.md · Demo 7



WORKED EXAMPLE

5 min · ask the room, then advance

The AI spend incident

WHAT YOU KNOW

Monday morning. Your support copilot's bill came in at 4.2× forecast.

12,000 conversations per week

Average 6 tool calls per conversation

18,000 input tokens per call — full case plus 8 similar cases

Near-frontier model · no caching · no cost owner named

You do not know whether the tool calls are legitimate or a retry loop, what the task success rate is, or whose budget this is.

THE QUESTION

You brief the CFO in ten minutes. What do you say?

The instinct is to promise a fix. The better answer names what you cannot yet know — and puts a cap around it.

Take two answers from the room before you advance. **The wrong answer is the teaching moment.**

The AI spend incident, answered

Two cuts you can ship this week, one change this quarter, and one honest admission

THIS WEEK

–30%

Turn on prefix caching and reorder the prompt. The case file and tool schemas are stable; nothing about the product changes.

THIS WEEK

–45%

Cut the 8 similar cases to 3 reranked. 18,000 input tokens becomes about 7,000, and the eval set says groundedness holds.

THIS QUARTER

Router

Six tool calls per conversation is the real driver. Route the routine intents down-tier and cap agent steps at the gateway.

MONDAY

1 metric

Cost per resolved ticket, with a ceiling — and a named owner on the budget. Not tokens. Not monthly spend.

And the honest part: "I do not know yet whether those six tool calls are real work or a retry loop."

"Here is the instrumentation that will tell us by Friday, and here is the per-session cap that protects us until then." A CFO will accept an unknown with a date and a cap attached. They will not accept a confident number that turns out to be wrong twice.

“

Full automation will be prohibitively expensive for most organizations. Leading organizations will use AI to drive customer engagement rather than to cut costs.

Patrick Quinlan, Senior Director Analyst, Gartner (January 2026) — projecting GenAI cost per resolution above \$3 by 2030, exceeding many offshore human agent costs

The mature version of this discipline is not "make the token cheaper." It is **"is this the right unit economics at all?"**

The 30-60-90 day plan

Cumulative 50–70% reduction on an unoptimised workload, with quality held — in this order

DAYS 1–30 · SEE THE SPEND

- Gateway in front of every LLM call
- Tag every call: BU, use case, env, model, session
- One dashboard: spend × model × team × use case
- One alert: daily spend over a threshold
- Find your top-5 most expensive call sites

\$0–\$200/yr on open source. Visibility on 100% of spend — which alone changes behaviour.

DAYS 31–60 · STOP THE WASTE

- Prompt caching on every stable prefix
- Reorder: static first, dynamic tail last
- Cut retrieval to a reranked handful
- Semantic cache on the top-10 repeated queries
- Cap output length; require structured outputs
- Move floor and volume traffic down-tier

Expected: 30–50% reduction, proven by eval.

DAYS 61–90 · GOVERN & FORECAST

- Three-tier router, eval gate in the same PR
- Per-team and per-user hard budget caps
- Circuit breakers: steps, tokens, wall-clock
- Cost per completed task, with a ceiling
- Best / base / worst quarterly forecast
- First joint FinOps review with Finance

Expected cumulative: 50–70% on the same workload.

The order is not negotiable. You cannot prove a saving you could not see, and you cannot govern a system you have not instrumented.

Where the savings come from

Every lever from today, with its honest range and the effort it costs

HOURS OF EFFORT

- Provider prompt caching — ~30% of the total bill
- Prompt reordering — unlocks the above
- Output caps and structured outputs — 20–40% of output
- Batch API for offline work — flat 50%
- Manual system-prompt compression — 20–40%
- Deleting unused tool schemas — often the largest line

DAYS TO WEEKS

- Retrieval right-sizing: reranked, not raw — 30–50% of input
- Chunking at 512 tokens, recursive — zero model calls
- Semantic caching — 40–80% of cacheable traffic
- Algorithmic compression — safe components only
- Compaction and note-taking — bounds multi-turn growth

WEEKS TO A QUARTER

- Tier routing — ~30% on a calibrated production cascade
- Distillation for high-volume narrow tasks
- Sub-agents and just-in-time retrieval
- Circuit breakers and caps — prevents rather than reduces
- Visibility and showback — 20–40% from behaviour alone

These **stack multiplicatively, not additively** — but each one needs its own eval gate, or you will not know which one broke the product.

Five things to take home

If you remember nothing else from these four hours

- 01 **Measure first. Optimise second.** Visibility alone produces 20–40% through behaviour change, before you write a line of optimisation code.
- 02 **The biggest lever is a dropdown.** A ~100× spread across tiers means model selection outweighs every other technique combined. Route deliberately.
- 03 **Agent cost is input cost.** Agents re-read accumulated context at every step — 62% of the agent bill. Context engineering is where agent bills are won.
- 04 **Divide by the success rate.** Cost per completed task, including cleanup. Reliability is a cost lever, and cleanup frequently exceeds the API bill.
- 05 **FinOps is a practice, not a project.** Tags, caps, circuit breakers and a weekly forecast-versus-actual review. The savings regress the moment the loop stops.

THE ONE-LINE VERSION

Cheap models,
small context,
and a meter
beat a smart model
with no owner.

You now have the arithmetic, the techniques and the controls. **The only thing left is the first measurement.**

YOUR 48-HOUR COMMITMENT

Put a meter on your biggest LLM call site before Friday.

Not a router. Not a compression pipeline. Not a governance framework.

One gateway, in front of your single largest call site, tagging every request with team, use case and session. Then open the dashboard on Friday afternoon.

You will find a number that surprises you — **and that number is where your first 30% is hiding.**

Appendix: the rate card

Per 1M tokens, standard tier — a snapshot, not a constant

Tier	Representative model	Input	Output	Cache read	Batch
Floor	DeepSeek V4-Flash class	\$0.14–0.22	\$0.28–0.66	~98% off	—
Floor	GPT-5.6 Luna class	\$0.20	\$1.20	90% off	50% off
Volume	Gemini 3.5 Flash-Lite class	\$0.30	\$2.50	—	50% off
Volume	Gemini 3.8 Flash class	\$0.75	\$3.75	90% off	50% off
Volume	Claude Haiku 4.5 class	\$1.00	\$5.00	90% off	50% off
Near-frontier	Claude Sonnet 5 class	\$2–3	\$10–15	90% off	50% off
Near-frontier	GPT-5.6 Terra class	\$2.00	\$12.00	90% off	50% off
Frontier	Claude Opus 5 class	\$5.00	\$25.00	90% off	50% off
Frontier	GPT-6 Astra class	\$10.00	\$50.00	90% off	50% off
Frontier	Claude Fable 5 / Mythos 5 class	\$10.00	\$50.00	97.5% off	50% off

Do not present these digits without checking them that morning. DeepSeek moved to peak/off-peak billing in August 2026 and Gemini 3.x Flash promotional pricing expires 31 December 2026 — a real budget event. The ratios on slide 14 do not move.

Published provider rate cards, September 2026: platform.claude.com, developers.openai.com, ai.google.dev, deepseek.ai.